# RAG集成：打造智能知识库

- 来源: https://class.imooc.com/lesson/5283#mid=105033
- 阶段: 阶段四: 【框架应用】高效AI开发
- 周: 第8周 LangChain框架入门
- 模块: RAG集成：打造智能知识库
- 课节: 图文 1-3 RAG集成：打造智能知识库

## 1-3 RAG集成：打造智能知识库

## RAG集成：打造智能知识库

> [!TIP] 阅读指南 第 3 章我们手动实现过 RAG 系统。本节用 LangChain 封装的 RAG 组件重新实现，只需更少的代码就能完成文档加载、分块、向量化和检索。 配套源码：samples/chapter8/langchain_rag/

## 回顾手动 RAG 的痛点

第 3 章实现游戏知识问答时，需要手动写文档分块、向量化、存库、检索、拼接 Prompt 等步骤，每个环节都要自己实现。换个向量模型或数据库，就要改多处代码。

LangChain 的 RAG 组件提供了统一的抽象，把常见操作封装成可插拔的模块。

## 四个核心组件

Document Loaders 用统一接口加载各种格式文档：

Document Loaders

```
from langchain.document_loaders import TextLoader
loader = TextLoader("game_guide.txt")
docs = loader.load()
```

Text Splitter 智能分块，保持语义完整：

Text Splitter

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)
```

Embeddings + VectorStore 统一接口操作向量化和数据库：

Embeddings + VectorStore

```
from langchain_community.embeddings import DashScopeEmbeddings
from langchain_community.vectorstores import Chroma

embeddings = DashScopeEmbeddings(model="text-embedding-v2")
vector_db = Chroma.from_documents(
    documents=chunks, embedding=embeddings,
    persist_directory="./chroma_db"
)
```

Retriever 灵活配置检索策略：

Retriever

```
retriever = vector_db.as_retriever(search_kwargs={"k": 3})

1
```

# 实战：完整 RAG 问答

```
# 简化后的完整流程
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vector_db.as_retriever()
)
answer = qa_chain.invoke({"query": "如何使用装饰器?"})
print(answer['result'])
```

相比手动实现的关键优势：所有组件（文档加载器、切分器、Embedding 模型、向量数据库）都可以按配置切换，代码无需改动。

自定义 Prompt 模板也很简单，在创建链时传入即可：

```
template = """参考以下文档回答问题：

{context}
问题：{question}
回答："""

prompt = PromptTemplate(template=template, input_variables=["context", "question"])
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vector_db.as_retriever(),
    chain_type_kwargs={"prompt": prompt}
)
```

## ■ 学点英语

| 中文 | English | 音标 | 说明 |
| --- | --- | --- | --- |
| 文档加载器 | Document Loader | /ˈdɑːkjumənt ˈloʊdər/ | 用统一接口加载各种格式文档的组件 |
| 文本分割器 | Text Splitter | /tekst ˈsplɪtər/ | 智能切分长文档为语义完整块的组件 |
| 检索器 | Retriever | /rɪˈtriːvər/ | 从向量数据库中检索相关文档的组件 |
| 检索问答链 | RetrievalQA Chain | /rɪˈtriːvəl kjuː eɪ tʃeɪn/ | 自动完成“检索→上下文拼接→回答”的链 |

## 下一节预告

Agent 能查天气、搜文档，但它的执行过程像"黑盒"，很难精确控制。如果想让 AI 必须先执行 A 步骤，只有满足条件才走 B 步骤，该怎么办？

这就是第 7 章 LangGraph 要解决的问题。LangGraph 用图（有向图）来建模工作流，天然支持条件分支、循环重试、人工介入和状态持久化——这些正是 create_agent 这类"黑盒 Agent"做不到的能力。